

Thin films in production with extended anisotropic kernels

Paul Kilgo
Blizzard Entertainment
Irvine, CA, USA
pkilgo@blizzard.com

Jamie Pilgrim
Blizzard Entertainment
Irvine, CA, USA
jpilgrim@blizzard.com

Ashraf Ghoniem
Blizzard Entertainment
Irvine, CA, USA
aghoniem@blizzard.com

Parag Havaldar
Blizzard Entertainment
Irvine, CA, USA
phavaldar@blizzard.com



Figure 1: Two views of a fluid drop simulation using anisotropic kernels, our automatic scaling correction, and a metaball implicit function.

ABSTRACT

Particles offer a convenient way of modeling fluid simulations, but ultimately the goal in photorealistic VFX is to extract a surface representing the fluid which looks true-to-life. A characteristic of dynamic fluid simulations is thin fluid sheets which are challenging to extract a surface from. Bumpy artifacts are frequently exposed in these features. Anisotropic smoothing kernels [Yu and Turk 2013] are a method to combat this, but they also present their own usability challenges. Similar ideas have been used before in the film industry [Museth et al. 2007].

This paper extends the anisotropic kernels technique to provide an automatic scaling correction feature so artists may work in normalized terms. We also use a metaball implicit function over smoothed-particle hydrodynamics (SPH) kernels to achieve smooth surfaces. With these extensions, artists can quickly create high quality surfaces with fewer particles.

ACM Reference Format:

Paul Kilgo, Jamie Pilgrim, Ashraf Ghoniem, and Parag Havaldar. 2022. Thin films in production with extended anisotropic kernels. In *Special Interest Group on Computer Graphics and Interactive Techniques Conference Talks (SIGGRAPH '22 Talks)*, August 07-11, 2022, Vancouver, BC, Canada. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3532836.3536264>

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

SIGGRAPH '22 Talks, August 07-11, 2022, Vancouver, BC, Canada

© 2022 Copyright held by the owner/author(s).

ACM ISBN 978-1-4503-9371-3/22/08.

<https://doi.org/10.1145/3532836.3536264>

1 HOUDINI IMPLEMENTATION

We implemented the anisotropic kernel approach using the Houdini Development Kit as two surface operator (SOP) nodes. One node is responsible for performing the principal component analysis (PCA) step [Yu and Turk 2013] and assigning each particle a transform point attribute. A second node consumes the transform and point data, and computes an implicit function using the point position data, outputting the result as either an OpenVDB [Museth 2013] level set or fog volume. The second node can switch the kind of implicit function it computes, toggling between the SPH smoothing kernel approach described in the original article, or a metaball-style blending of transformed sphere level sets.

2 ANISOTROPY IMPROVEMENTS

The singular value decomposition (SVD) step of PCA is expensive and deters interactivity of the node. We've designed our SOP to only recompute the PCA step when the neighborhood size r_i changes. When changing other parameters, like the two particle scaling parameters k_s and k_n , we are free to reuse the existing PCA computation.

Using Houdini's instancing attribute rules [SideFX 2022] means highly optimized nodes like "Copy to Points" can be used to pre-visualize the surface with instanced rough geometry. We output the computed transform as a point attribute transform which allows more open-ended experimentation with this node's output. Different from the original paper, we store a forward transform T_i at this attribute since it is what Houdini expects. The second surfacing SOP inverts this matrix to compute the implicit function.

The size of the neighborhood search radius affects the overall scale of particles, leading to initial difficulties getting scale of the

surface right. In [Yu and Turk 2013] a scaling parameter k_s is used to correct for this. The approach is to take a particle’s covariance matrix C and choose a k_s such that $||k_s C|| \approx 1$ where $||\cdot||$ is a matrix determinant. That is then used to compute a corrected diagonal eigenvalue matrix $\tilde{\Sigma}$, acting as a uniform scale for the particle’s transformation matrix. One strategy is to automatically compute k_s according to that criteria,

$$k_s = \sqrt[3]{\frac{1}{||C||}}. \quad (1)$$

We introduce a simple correction factor based on the computed eigenvalues of PCA. Because σ_1 is the first component’s eigenvalue as determined by PCA, it is larger than the other eigenvalues. Since the eigenvalues are used as the scaling terms of a transformation matrix, we can invert σ_1 to find a correction factor s_i for the current particle,

$$s_i = 1/\sigma_1. \quad (2)$$

Using s_i as a correction factor per-particle leads to larger particles especially at the corners of the bounding box. Instead, we opt to smooth the correction factor over the neighborhood. For each point, compute the neighborhood of particles. Find the mean of the correction factors for the neighborhood, $\bar{s} = (1/N)\sum_i s_i$ where N is the size of the neighborhood.

Finally, we use $\bar{s}$ as a scaling transform to correct the transform of the particle,

$$T_i = h_i \cdot \bar{s} I \cdot R \cdot \tilde{\Sigma} \cdot R^T, \quad (3)$$

where h_i is the smoothing radius of the particle, R is the rotation matrix, and $\tilde{\Sigma}$ is a diagonal matrix of the modified eigenvalues as defined in [Yu and Turk 2013]. The result is the first principal component of each of the particles will be roughly identical in scale. This is useful because the scale of the resulting volumetric primitives is set by the artist, and being able to freely adjust the neighborhood radius without affecting scale makes for more predictable results.

3 SURFACING

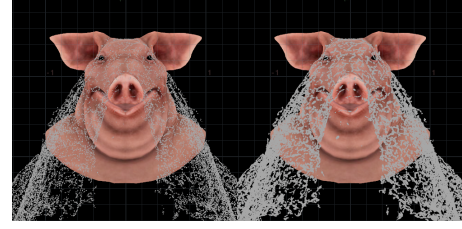
Once the particle transform is computed, the final step is to extract the fluid surface defined by the isovalue of a chosen implicit function. We implement this step in a second SOP using OpenVDB.

The first step is to compute the active voxels defined by the particles. To do this, we compute a bounding box defined by each particle. By default, each particle has a unit sphere’s bounding box, taking the particle’s transform into account, though the artist can set the radius of the sphere to change the size of the bounding box. Each voxel contained in a bounding box is marked active.

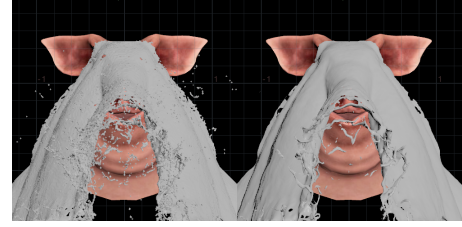
Once the active voxels are known, we iterate over them to sample the implicit function, $\Phi(\mathbf{x})$. The original paper uses an implicit function dependent on particle mass, density, and smoothing kernels defined in [Müller et al. 2003]. One can also apply the transform to more elementary implicit functions. We also support using a metaball blending of sphere level set functions over the neighborhood,

$$\Phi(\mathbf{x}) = \sum_i \exp(1 - |T_i^{-1} \cdot (\mathbf{x} - \mathbf{p}_i)|) - c, \quad (4)$$

where $\mathbf{p}_i$ is the position of the i -th particle, and c is a user parameter. If a voxel has no neighbors, it is set to inactive. This function describes a smooth surface with fewer artifacts compared to the SPH smoothing kernels, especially on a smaller particle count.



(a) Left: Transforms of anisotropic kernels visualized with spheres, with $k_s = 4000$. Right: Same transforms with $k_s = 3$ and our automated scale correction applied. The automatic corrections allows artists to work in normalized terms to find the desired scale.



(b) Left: A reduced point count surface is reconstructed using the W_{poly6} SPH kernel [Müller et al. 2003]. Right: The same point set using the metaball implicit function.

Figure 2: Extensions to the anisotropic kernels technique.

4 CONCLUSION

We present extensions to the anisotropic kernels technique for modeling thin sheets of fluid. One extension decouples the nearest neighborhood search radius from the overall scale of particle contributions, which allows artists to more quickly arrive at the desired scale. We offer implementation considerations for the Houdini Development Kit. We propose a metaball implicit function for blending sphere level sets. The extensions build on the existing approach to improve its user experience and create high quality, smooth, and thin surfaces without adding more particles into simulation.

ACKNOWLEDGMENTS

The authors would like to thank AJ Zalinka, Chris Allen, Rafael Campos, and the FX and R&D departments for their help and feedback on this project.

REFERENCES

- Matthias Müller, David Charypar, and Markus Gross. 2003. Particle-Based Fluid Simulation for Interactive Applications. In *Proceedings of the 2003 ACM SIGGRAPH/Eurographics Symposium on Computer Animation* (San Diego, California) (SCA '03). Eurographics Association, Goslar, DEU, 154–159.
- Ken Museth. 2013. VDB: High-Resolution Sparse Volumes with Dynamic Topology. *ACM Trans. Graph.* 32, 3, Article 27 (Jul 2013), 22 pages. <https://doi.org/10.1145/2487228.2487235>
- Ken Museth, Michael Clive, and Nafees Bin Zafar. 2007. Blobscatter: Surfacing Particle System in “Pirates of the Caribbean 3”. In *ACM SIGGRAPH 2007 Sketches* (San Diego, California) (SIGGRAPH '07). Association for Computing Machinery, New York, NY, USA, 20–es. <https://doi.org/10.1145/1278780.1278804>
- SideFX. 2022. Copying and instancing point attributes. <https://www.sidefx.com/docs/houdini/copy/instanceattrs>. (Accessed on 01/25/2022).
- Jihun Yu and Greg Turk. 2013. Reconstructing Surfaces of Particle-Based Fluids Using Anisotropic Kernels. *ACM Trans. Graph.* 32, 1, Article 5 (Feb 2013), 12 pages. <https://doi.org/10.1145/2421636.2421641>